

CSA0201 – C PROGRAMMING

**Smart Emergency Resource Management and Analysis
System using C**

NAME: C. Rushitha, D. Thanvika, T. Vijay Kumar, K. Vinay, A. Ajay

REGNO: 192320084, 192320087, 192320095, 192312080, 192320038

PSEUDOCODE / ALGORITHM DESIGN

This section presents the overall system algorithm and the algorithms for the major operations — searching, sorting, merging, and resource analysis.

1. Main Menu Driver

```
BEGIN
  LOAD resources from file
  REPEAT
    DISPLAY menu (13 options)
    READ choice
    SWITCH(choice)
      CASE 1: CALL addResource()
      CASE 2: CALL updateResource()
      CASE 3: CALL displayAll()
      CASE 4: CALL searchMenu()
      CASE 5: CALL sortMenu()
      CASE 6: CALL mergeDepartments()
      CASE 7: CALL findDuplicates()
      CASE 8: CALL analyseResources()
      CASE 9: CALL displayCriticalResources()
      CASE 10: CALL generateReport()
      CASE 11: CALL saveToFile()
      CASE 12: CALL loadFromFile()
      CASE 13: SAVE and EXIT
      DEFAULT: PRINT "Invalid choice"
    UNTIL choice = 13
  END
```

2. Add / Update Resource

```
FUNCTION addResource()
  READ id, name, category, department, quantity, threshold, priority
  IF searchByID(id) EXISTS THEN
    PRINT "Duplicate ID" AND RETURN
  ENDIF
  APPEND record to resources[]; count = count + 1
END FUNCTION
```

```
FUNCTION updateResource()
  READ id; idx = searchByID(id)
  IF idx = -1 THEN PRINT "Not found" AND RETURN
  r = POINTER TO resources[idx]
  READ field choice; UPDATE r->field with new value
END FUNCTION
```

3. Searching (Linear + Recursive Binary Search)

```
FUNCTION searchByID(arr, n, id)
  FOR i = 0 TO n-1
    IF arr[i].id = id THEN RETURN i
  RETURN -1
END FUNCTION
```

```
FUNCTION recursiveBinarySearchByID(arr, low, high, id)
  IF low > high THEN RETURN -1
  mid = (low + high) / 2
  IF arr[mid].id = id THEN RETURN mid
  ELSE IF arr[mid].id > id THEN
    RETURN recursiveBinarySearchByID(arr, low, mid-1, id)
  ELSE
    RETURN recursiveBinarySearchByID(arr, mid+1, high, id)
END FUNCTION
```

4. Sorting (Quantity / Priority / Department)

```
FUNCTION bubbleSortByQuantity(arr, n)
  FOR i = 0 TO n-2
    FOR j = 0 TO n-i-2
      IF arr[j].quantity > arr[j+1].quantity THEN SWAP(arr+j, arr+j+1)
    END FUNCTION
  END FUNCTION
```

```
FUNCTION selectionSortByPriority(arr, n)
  -- similarly, using minIdx and a pointer-based swap
END FUNCTION
```

```
FUNCTION sortByDepartment(arr, n)
  -- bubble sort comparing department strings with strcmp
END FUNCTION
```

5. Merge Departments (with Recursive Duplicate Removal)

```
FUNCTION isDuplicate(arr, n, r)      -- RECURSIVE
  IF n <= 0 THEN RETURN FALSE
  IF arr[n-1].id = r.id THEN RETURN TRUE
  RETURN isDuplicate(arr, n-1, r)
END FUNCTION
```

```
FUNCTION mergeDepartments(dept1, dept2)
  FOR EACH resource IN resources[]
    IF resource.department = dept1 OR dept2 THEN
      IF NOT isDuplicate(merged, mergedCount, resource) THEN
        APPEND resource TO merged[]
  DISPLAY merged[]
```

END FUNCTION

6. Duplicate Identification (Recursive)

```
FUNCTION findDuplicatesRecursive(index, found)
  IF index >= count THEN RETURN      -- base case
  FOR j = index+1 TO count-1
    IF resources[index].id = resources[j].id THEN
      PRINT duplicate pair; found = found + 1
    CALL findDuplicatesRecursive(index+1, found) -- recursive case
  END FUNCTION
```

7. Resource Analysis & Critical Report

```
FUNCTION analyseResources()
  FOR EACH resource
    diff = quantity - threshold
    IF diff > 5 THEN adequate++
    ELSE IF diff >= 0 THEN low++
    ELSE critical++
  PRINT adequate, low, critical counts
END FUNCTION
```

8. Consolidated Report (Recursion + File Handling)

```
FUNCTION totalQuantityRecursive(arr, n)      -- RECURSIVE
  IF n <= 0 THEN RETURN 0
  RETURN arr[n-1].quantity + totalQuantityRecursive(arr, n-1)
END FUNCTION
```

```
FUNCTION departmentWiseTotalRecursive(depts, deptCount, index) -- RECURSIVE /
hierarchical
  IF index >= deptCount THEN RETURN
  total = SUM of quantity WHERE department = depts[index]
  PRINT department, total
  CALL departmentWiseTotalRecursive(depts, deptCount, index+1)
END FUNCTION
```

```
FUNCTION generateReport()
  OPEN report.txt
  WRITE total quantity, department-wise totals, critical list
  CLOSE file
END FUNCTION
```

9. File Handling (Save / Retrieve)

```
FUNCTION saveToFile()
  OPEN resources.txt in write mode
```

```
    FOR EACH resource: WRITE fields separated by '|'
    CLOSE file
END FUNCTION
```

```
FUNCTION loadFromFile()
    OPEN resources.txt in read mode
    IF file NOT found THEN RETURN      -- first run
    WHILE NOT end-of-file AND count < MAX
        READ one record INTO resources[count]; count++
    CLOSE file
END FUNCTION
```